Software Development Life Cycle (SDLC)

Contents

Purpose of the SDLC 2
Main Stages of the SDLC 2
Feasibility Study 2
Requirements Analysis 10
Design 22
Implementation 29
Testing 37
Deployment 50
Maintenance 57
Methodologies 67

Purpose of the SDLC

The Software Development Life Cycle, or SDLC, is a structured process used to guide software development from the initial idea all the way to maintenance and retirement.

Its main goal is to produce high quality software quickly and efficiently by breaking development into clear stages.

A good SDLC helps teams:

- Plan work properly
- Reduce risk
- Improve quality
- Manage cost and time
- Maintain software after release

Main Stages of the SDLC

The SDLC usually follows these broad stages:

Feasibility Study

Requirements Analysis

Design

Implementation

Testing

Deployment

Maintenance

Each stage has a different purpose, but together they form a complete development process.

Feasibility Study

What it is

A feasibility study happens early in a project when a business need, problem, or opportunity has been identified.

Its purpose is to decide whether the proposed system is practical, worthwhile, and achievable before significant time and money are spent.

This stage is usually short and focused.

Why it matters

A feasibility study helps an organisation avoid wasting resources on projects that:

- do not support business goals
- cannot be delivered on time or within budget
- are too risky
- cannot integrate with existing systems

In simple terms, it asks:

> **"Should we do this project at all?"**

Key questions in a feasibility study

A feasibility study should answer questions such as:

- Does the system support the overall objectives of the organisation?
- Can it be implemented within schedule and budget?
- Can it be built using current technology and available resources?
- Can it integrate with existing systems?

If the answer to any of these is clearly no, then the project may not be worth pursuing.

Key outcomes

A feasibility study helps to:

- identify new opportunities
- narrow down business alternatives
- support decision making
- document that the idea was properly investigated
- reduce unnecessary business risk

Business Case

What it is

The business case explains why the project should go ahead. It focuses on the value the project brings to the organisation and justifies the investment.

Why it matters

The business case is important because it links the software project to business goals. It helps decision makers understand:

- the problem being solved
- the expected benefits
- the likely costs
- the risks involved
- whether the project is worth funding

Types of benefits

Benefits may be:

Tangible benefits

These can be measured clearly, often in money or time.

Examples:

- reduced staffing costs
- faster processing times
- increased revenue
- lower support costs

Intangible benefits

These are harder to measure directly but still valuable.

Examples:

- better customer satisfaction
- stronger company reputation
- improved staff morale
- better decision making

Types of Feasibility

A feasibility study looks at different factors that affect whether a project can succeed.

These often include:

- economic
- technical
- legal
- operational
- environmental

Technical Feasibility

What it is

Technical feasibility checks whether the organisation has the technology, hardware, software, and skills needed to build and run the proposed system.

What it considers

It looks at:

- whether the current hardware and software are suitable
- whether existing systems can be upgraded

- whether the development team has the required technical skills
- whether the chosen technology is stable and well established
- whether support is available for that technology

A project may sound great on paper, but if the team cannot build it, the project is cooked before it leaves spawn.

Common technical risks

- inability to construct the system
- poor system performance
- cost overruns
- time overruns
- failure to achieve expected benefits
- integration problems with existing hardware or software

Economic Feasibility

What it is

Economic feasibility looks at whether the expected benefits outweigh the expected costs. This is often done using cost-benefit analysis.

What it considers

Economic analysis measures:

- development costs
- operational costs
- maintenance costs
- expected financial gains
- expected savings
- return on investment

Cost-benefit analysis

Cost-benefit analysis compares the strengths and weaknesses of different options to find the best choice. A system is considered economically feasible if expected benefits are equal to or greater than expected costs.

Why it matters

Even if a system is technically possible, it may still not be worth building if it costs too much for too little return.

Legal Feasibility

What it is

Legal feasibility checks whether the proposed system complies with laws and regulations.

Examples

This may include:

- GDPR and data protection law
- copyright law
- computer misuse law
- licensing rules
- industry regulations

Why it matters

A system that breaks legal rules can expose the organisation to fines, legal action, and reputational damage.

Operational Feasibility

What it is

Operational feasibility looks at whether the proposed system will work in the real business environment.

It asks whether users, staff, and processes can support the system in practice.

Key idea

A technically brilliant system is useless if nobody wants to use it or if it does not fit how the business operates.

PIECES framework

Operational feasibility is often analysed using **PIECES**:

- **Performance**
  Does the system improve speed, capacity, or response times?
- **Information**
  Does it improve the quality, accuracy, or availability of information?
- **Economy**
  Does it reduce costs or make better use of resources?
- **Control**
  Does it improve security, monitoring, and error handling?
- **Efficiency**
  Does it make processes less wasteful and more productive?
- **Services**
  Does the system meet the needs of the users and organisation?

Environmental Feasibility

What it is

Environmental feasibility considers the impact of the system on people and the environment.

Examples

It may include:

- energy use
- server and infrastructure costs
- sustainability concerns
- environmental impact of running the application
- human factors such as usability and working conditions

Risk Analysis

What it is

Risk analysis identifies possible problems that could affect the project and estimates their consequences.

Why it matters

Every project carries risk, so teams should identify and assess risks continuously, not just once at the start.

Common sources of risk

Risks can come from:

- use of new or untested technology
- users resisting change
- lack of critical resources
- competition in the market
- inexperienced team members
- poor understanding of the business area

Key point

The goal is not to eliminate all risk. The goal is to understand it, manage it, and decide whether the project is still worth doing.

Considering Alternatives

What it means

Before committing to one solution, a team should investigate multiple alternatives.

This includes different ways of:

- organising the business
- solving the problem
- positioning the product in the market
- delivering value to users

Why it matters

Exploring alternatives prevents teams from locking onto the first idea too early. A project should not go ahead just because it is possible. It should go ahead because it is the best realistic option.

Collection of Information

What it is

Software projects rely on good data. This includes information about:

- project progress
- development processes
- quality levels
- team performance
- risks and outcomes

Why it matters

The challenge is to collect enough useful information without creating extra burden for the development team.

Good practice

Data collection should:

- be based on clear metrics and models
- support project, process, and quality management
- be planned early
- be useful for improvement
- avoid becoming unnecessarily expensive or time consuming

Writing a Feasibility Report

After the feasibility study, findings are usually written into a report.

Common sections of a feasibility report

- Introduction / Executive Summary
- Background
- Outline of the project
- Methodology or method of analysis
- Overview of alternatives

- Conclusion
- Recommendation

Purpose of the report

The report provides evidence that the project was properly analysed and gives decision makers a clear basis for approving or rejecting it.

Key Takeaways

- The SDLC is a structured process for building software from idea to maintenance.
- Its purpose is to produce high quality software efficiently.
- The first major stage is the feasibility study, which decides whether a project is worth doing.
- Feasibility includes technical, economic, legal, operational, and environmental considerations.
- Risk analysis and considering alternatives are essential early activities.
- A business case justifies the project by explaining costs, benefits, and value.
- Later SDLC stages include requirements analysis, design, implementation, testing, deployment, and maintenance.

Requirements Analysis

What it is

Requirements Analysis defines what the system must do and what users need.

It is about:

- understanding problems
- defining solutions clearly
- aligning stakeholders before development starts

If this stage is weak, everything after it breaks.

Roles: Customer vs User

Customer

- Pays for or commissions the system
- Focuses on business goals and value
- Cares about outcomes such as ROI and efficiency

User

- Uses the system day to day
- Focuses on usability and functionality
- Cares about experience and practicality

Key Idea

- Customers define what is needed
- Users define how it should work in practice

Good requirements combine both perspectives.

Why Stakeholder Involvement Matters

- Defines real problems rather than assumed ones
- Builds agreement early
- Improves estimation of time, cost, and scope
- Reduces risk of failure
- Ensures future users are considered

Requirements describe reality as it should be, not as it is.

User Research

Purpose

To understand:

- who users are
- what problems they face
- how they currently work

Key Rule

Users must confirm and agree before moving forward.

How User Research Helps

- Aligns stakeholders on system goals
- Validates real needs early
- Reduces risk of building incorrect features
- Improves usability

Use Cases

- Describe behaviour from the user's perspective
- Capture real interactions
- Reduce ambiguity

Information Gathering Techniques

- Questionnaires
- Interviews
- Contextual enquiry
- Focus groups
- Personas
- Customer journey mapping

Personas

What they are

Fictional user profiles based on research.

Why they matter

- Represent different user types
- Highlight varying needs and expectations
- Prevent biased design decisions

Customer Journey Mapping

What it is

A visual representation of the user's experience over time.

Sample customer journey map

Purpose

- Understand interactions
- Identify pain points
- Improve experience

Core Steps

1. Define persona
2. Understand goals
3. Map touchpoints
4. Identify pain points
5. Fix issues

6. Continuously improve

Functional vs Non-Functional Requirements

Functional Requirements

Define what the system does.

Examples:

- user login
- payment processing
- data storage

Non-Functional Requirements

Define how the system performs.

Examples:

- speed
- scalability
- reliability
- security

Common Non-Functional Requirement Types

- Security
- Availability
- Reliability
- Performance
- Capacity
- Continuity
- Supportability
- Maintainability
- Scalability

Key Non-Functional Concepts

Security

- Ensures confidentiality, integrity, and availability
- Requires secure development practices
- Must be integrated throughout the SDLC
- Involves people, processes, and technology

Availability

- Probability the system is operational
- Focus on minimising downtime

Reliability

- Probability of operating without failure over time
- Focus on consistency and failure rate

Important:

- Does not include repair time
- Measures time until failure

Performance

- Defines what the system must do and how well it must do it
- Derived from functional requirements (quantified behaviour)
- Identified through functional analysis

Performance requirements analysis:

- Identifies performance objectives during requirements gathering

---

Capacity

- Measures how much workload a system or process can handle

Examples:

- number of stories completed
- number of sprints delivered

Key idea:

- Determines what can be delivered within time, cost, and effort constraints

---

Continuity

- Ability of the system to continue operating
- Defines how services are maintained or restored

Includes:

- resources needed for recovery
- uptime expectations
- system stability over time

Supportability

- Ability to support the system across its lifecycle
- Includes maintenance, logistics, and operational readiness
- Must be achievable at an acceptable cost

Maintainability (Serviceability)

- Ease of maintaining and repairing the system
- Focus on early detection of issues
- Some systems can self-correct before failure

Scalability

- Ability to handle increased workload without bottlenecks

Types of Scalabilities

Physical

- Infrastructure capacity
- Storage, bandwidth, hardware

Intangible

- Business growth capability
- Supporting new products, brands, or processes

Expressing Requirements

Common formats:

- written descriptions
- use cases
- user stories
- diagrams (UML, flowcharts)
- prototypes

Goal: requirements must be clear, testable, and shared.

Functional Decomposition

- Breaks complex problems into smaller tasks
- Focuses on operations rather than data relationships

Example (ATM):

- Insert card
- Enter PIN
- Process transaction

System view:

- card reading
- PIN verification
- transaction processing

A functional requirements document represents this breakdown in text form.

UML (Unified Modelling Language)

- Standard modelling language
- Defines 14 diagram types

- Represents:
    - structure
    - behaviour
    - interactions

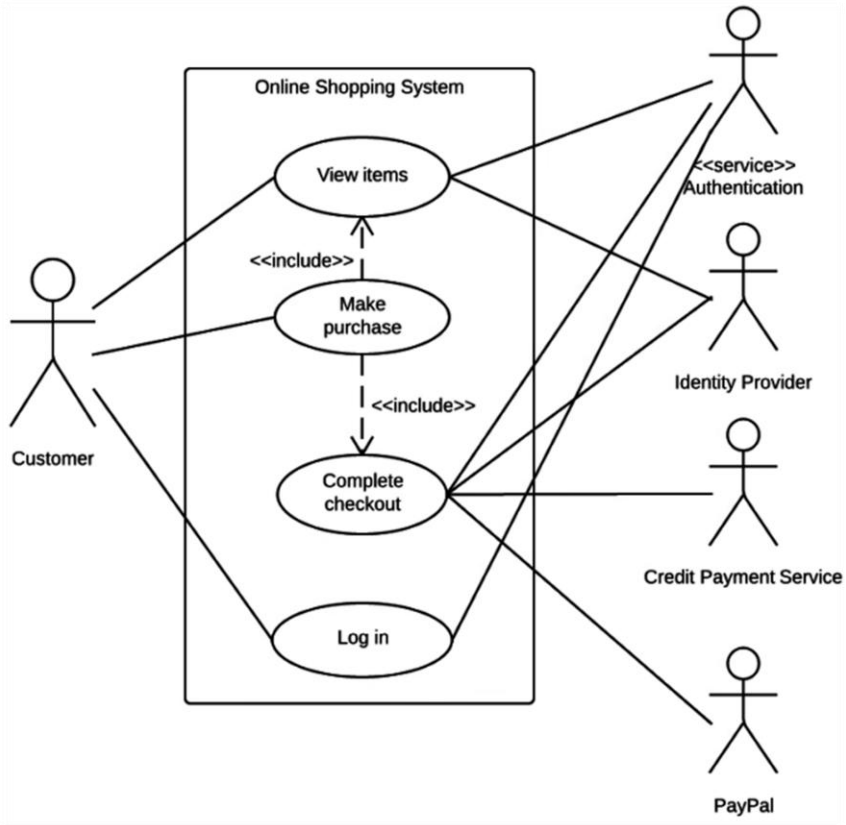
Used to visualise object-oriented systems.

Use Case Diagrams

- Show system functionality
- Use actors and use cases
- Represent system behaviour

Activity Diagrams:

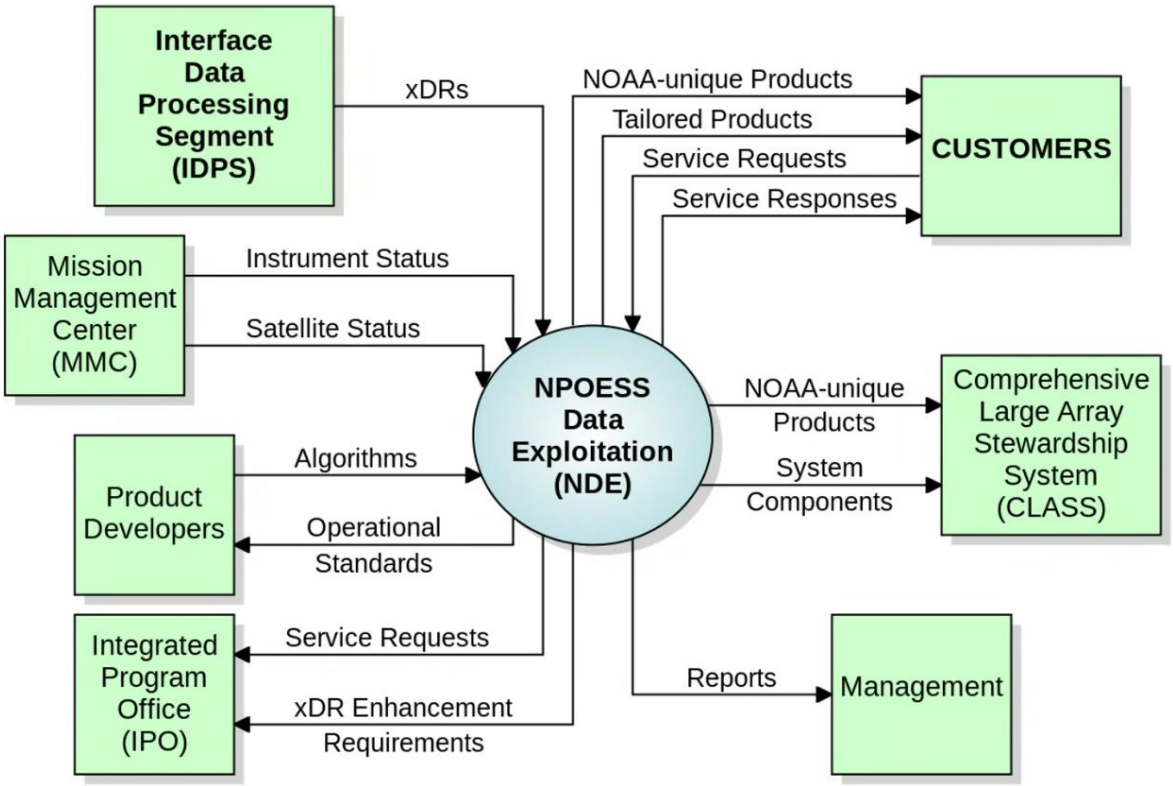
- Represent workflow or process flow
- Show sequence of operations



Context Diagram

- Defines system boundaries

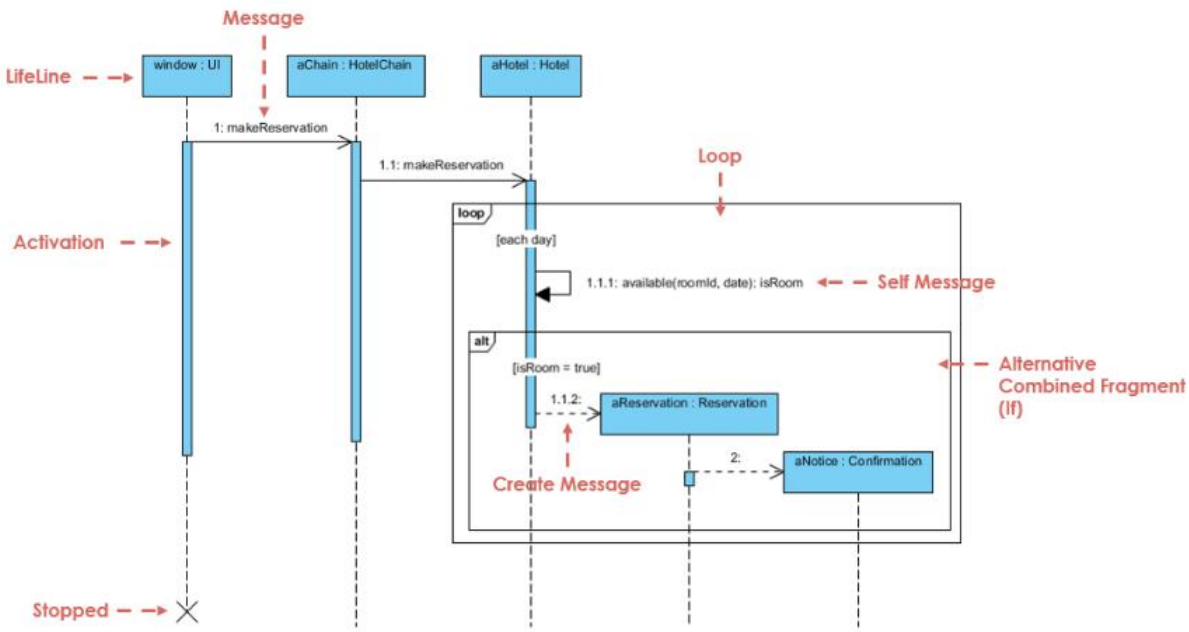
- Shows interactions with external entities



-

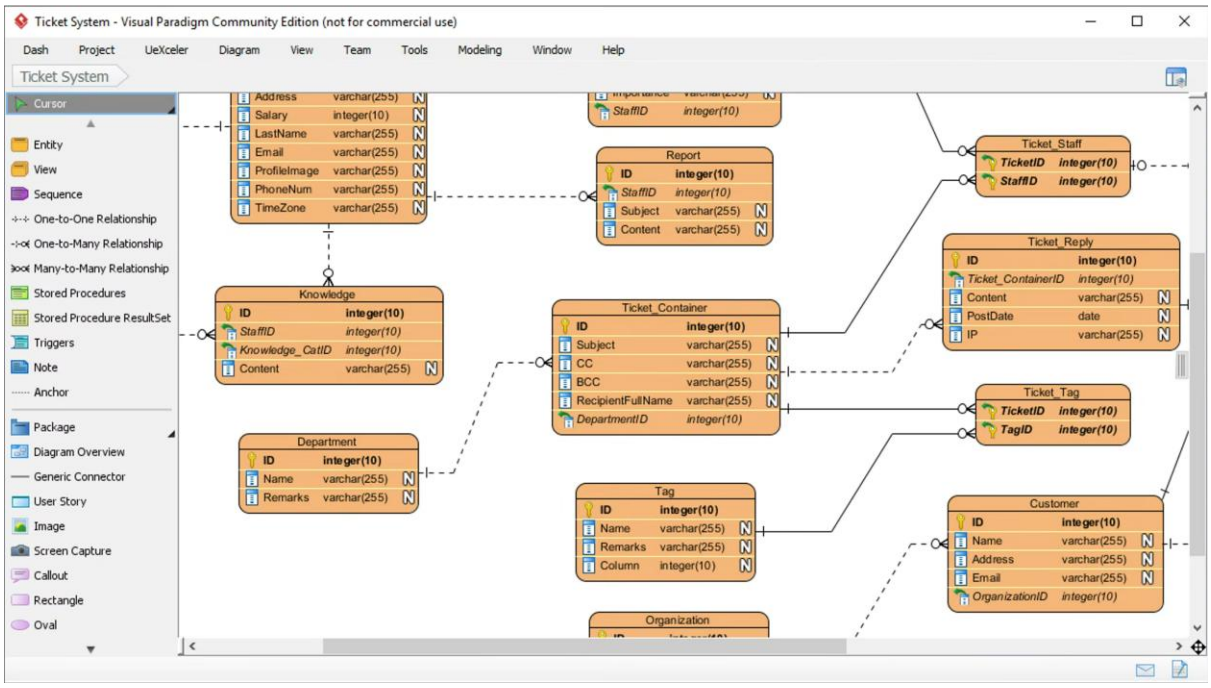
Sequence Diagrams

- Show interactions over time



Entity Relationship Diagrams

- Represent data relationships



Tools for making Diagrams

Visual Paradigm

Software - https://www.visual-paradigm.com/download/community.jsp

Diagrams.net

Website - https://app.diagrams.net/

Lucid Chart

Website - https://www.lucidchart.com/

Qualities of Good Requirements

Good requirements should be:

- Unambiguous
- Testable
- Clear
- Correct
- Understandable
- Feasible

- Independent
- Atomic
- Necessary
- Implementation-free

Impact of Poor Requirements

- Scope creep
- Increased cost and delays
- Poor user experience
- Rework
- System failure

Data Dictionary / Glossary

What it is

A structured definition of data elements used in the system.

Purpose

- Improve understanding
- Ensure consistency
- Make data reusable and interpretable

Includes

- variable names
- measurement units
- allowed values
- definitions
- optional synonyms
- descriptions

Key Takeaways

- Requirements Analysis defines what the system must do and what users need.
- Customers focus on business value, while users focus on practical use and usability.

- Stakeholder involvement is essential for agreement, accurate scope, and reducing project risk.
- User research helps analysts understand users, validate needs, and gain user buy-in.
- Common information gathering techniques include questionnaires, interviews, contextual enquiry, focus groups, personas, and customer journey mapping.
- Functional requirements define what the system does.
- Non-functional requirements define how well the system performs and under what constraints.
- Common non-functional requirements include security, availability, reliability, performance, capacity, continuity, supportability, maintainability, and scalability.
- Security should be considered throughout the SDLC and includes people, processes, and technology.
- Reliability measures time until failure, while availability measures whether the system is operational.
- Performance requirements define expected system behaviour in measurable terms.
- Capacity focuses on the workload a system or process can handle within limits.
- Continuity defines how operations can continue or recover after disruption.
- Supportability and maintainability focus on long-term operation, repair, and lifecycle support.
- Scalability is the ability to handle growth without structural bottlenecks, including both physical and non-physical growth.
- Requirements can be expressed using written descriptions, use cases, user stories, diagrams, and prototypes.
- Functional decomposition breaks a complex system into smaller tasks and operations.
- Common tools in requirements analysis include use case diagrams, context diagrams, activity diagrams, sequence diagrams, entity relationship diagrams, and UML.
- Good requirements should be unambiguous, testable, clear, correct, understandable, feasible, independent, atomic, necessary, and implementation-free.
- Poor requirements can lead to scope creep, delays, rework, wasted cost, poor user experience, and system failure.
- A data dictionary improves consistency and understanding by defining variables, meanings, units, and allowed values.

Design

Purpose and Scope

The purpose of a design document is to describe the system clearly enough for development to proceed with full understanding of:

- what needs to be built
- how it should be built

Design Stages

Preliminary Design

- Defines overall system architecture
- Defines data architecture
- High-level structure of the system

Detailed Design

- Defines detailed data structures
- Develops algorithms
- Specifies how components will be implemented

Functional vs Non-Functional Design

Functional Design

Focuses on making the system **fit for purpose**.

Includes:

- logical flow of the system
- inputs and outputs
- data organisation
- business and processing rules
- how the system appears to users

Non-Functional Design

Focuses on making the system **fit for use**.

Includes:

- performance
- scalability
- security
- reliability
- usability

Impact on Design

- Functional requirements define system behaviour and help identify missing features
- Non-functional requirements define performance and quality attributes

Examples of functional areas:

- transaction corrections and cancellations
- business rules
- reporting
- administrative functions
- authorisation levels
- audit tracking
- external interfaces
- legal and regulatory requirements

Examples of non-functional areas:

- scalability
- capacity
- availability
- reliability
- recoverability
- data integrity

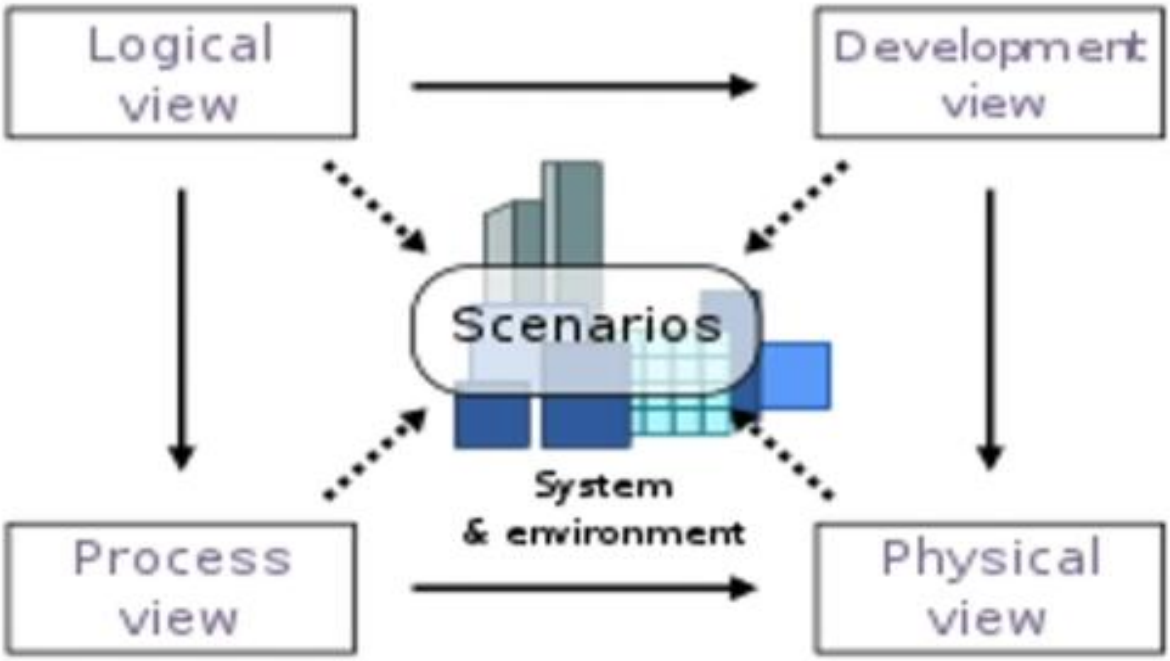
System Components to Consider

- Software architecture

- User interface
- User ergonomics
- Infrastructure and platform components
- Tools
- Operational processes
- Measures and metrics

Architecture

- Defines the fundamental structure of the system
- Includes components, relationships, and properties
- Determines how parts of the system interact



User Interface (UI)

- The point of interaction between user and system
- Includes input devices and interface design
- Must support usability and clarity

User Ergonomics

- Focuses on user-centred design
- Ensures systems are easy to use and efficient

Includes:

- user needs analysis
- interface design
- user support
- usability testing

Infrastructure and Platform Components

- Components are modular software units
- Each component has a defined interface
- Must follow consistent behaviour standards

Key concept:

- Separation of concerns
  Each component handles a specific responsibility

Tools

UML and Modelling Tools

- Rational Rhapsody
- Umbrello
- Visual Paradigm
- Lucidchart

Flowcharting Tools

- Visio
- Draw.io

Basic Tools

- Pencil and paper

Operational Processes

Lifecycle Processes

- Discover requirements
- Design solutions
- Construct solutions
- Validate solutions
- Deploy solutions
- Support and maintain systems

Cross-Lifecycle Processes

- Project planning
- Estimation of time, effort, and quality
- Tracking and reporting progress
- Defining standards and processes
- Monitoring performance
- Training
- Configuration management (version and release control)

Measures and Metrics

What they are

Software metrics are measurable characteristics of software.

Why they matter

- measure performance
- support planning
- track productivity
- evaluate quality

Characteristics of Good Metrics

- simple and computable

- consistent and unambiguous
- use standard units
- independent of programming language
- easy to adapt and calibrate
- cost-effective to collect
- accurate and reliable
- relevant to quality improvement

Scope of Metrics

- cost and effort estimation
- productivity measurement
- data collection
- quantity and size metrics
- reliability models
- performance evaluation
- complexity metrics
- capability maturity assessment
- management by metrics
- evaluation of tools and methods

Human Computer Interaction (HCI)

What it is

Study of how people interact with computer systems.

Components

- the user
- the computer
- the interaction between them

Why Good Design Matters

- Most critical phase of SDLC
- Improves usability and user satisfaction
- Increases competitiveness
- Builds trust through consistent experience
- Enhances interaction quality

Prototyping and Modelling

What it is

Building a model of a system to test ideas and design decisions.

Purpose

- improve usability
- validate requirements
- support iterative development

Advantages

- reduces development time
- reduces costs
- requires user involvement
- provides measurable feedback
- improves user satisfaction
- prepares users for final system
- highlights future improvements

Disadvantages

- may reduce depth of analysis
- users may expect prototype-level performance
- developers may become attached to prototypes
- risk of incomplete systems

- incomplete documentation
- advanced tools can reduce time savings

Key Takeaways

- The design stage translates requirements into a technical blueprint
- Preliminary design defines structure, detailed design defines implementation
- Functional design focuses on behaviour, non-functional design focuses on quality
- System design must consider architecture, UI, infrastructure, and processes
- Metrics and processes support quality and control
- HCI ensures systems are usable and effective
- Prototyping improves design but must be managed carefully

Implementation

Version Control

What it is

Version control is the process of tracking and managing changes to files over time.

It allows multiple developers to:

- work on the same project
- track changes
- collaborate safely

Change History and Change Management

Software Change Management

- Controls which changes are:
    - encouraged
    - allowed
    - prevented

Based on:

- cost

- schedule
- project priorities

Purpose

- identify origin of changes
- define decision points
- assign roles and responsibilities

Concurrent Working

What it is

Concurrency is when multiple operations occur in overlapping time periods.

In software development:

- multiple developers can work on the same files simultaneously
- each developer works on their own copy

Key Concepts

- Not the same as parallelism
- Requires coordination to avoid issues

Common Issues

- race conditions
- deadlocks
- conflicting changes

Version Control Systems (VCS)

- Track all changes to files
- Allow collaboration across time and location
- Enable multiple developers to work concurrently

Example:

- developers edit files locally
- changes are committed to a shared repository

Tools

Source Control Tools

- GitHub / GitLab
- Mercurial
- Apache Subversion
- CVS

Continuous Integration Tools

- Jenkins
- Hudson

Team Management Tools

- Jira
- GitHub / GitLab
- Eclipse
- ActiveCollab
- VersionOne

Preventing Conflicts

- Conflicts are normal and expected
- Can be constructive or destructive

Causes

- poor planning
- unclear roles
- overlapping changes

Prevention

- clear responsibilities
- defined goals
- good communication
- proper version control practices

Traceability

What it is

Ability to track work items across the development lifecycle.

Purpose

- monitor progress
- show history of changes
- support compliance

How it works

- links related items (for example requirements to test cases)
- shows dependencies between components

Security in Version Control

- Ensures confidentiality, integrity, and availability
- Requires secure development practices
- Built into the development process

Key Features

- revision history
- distributed copies of repositories
- accountability through user tracking

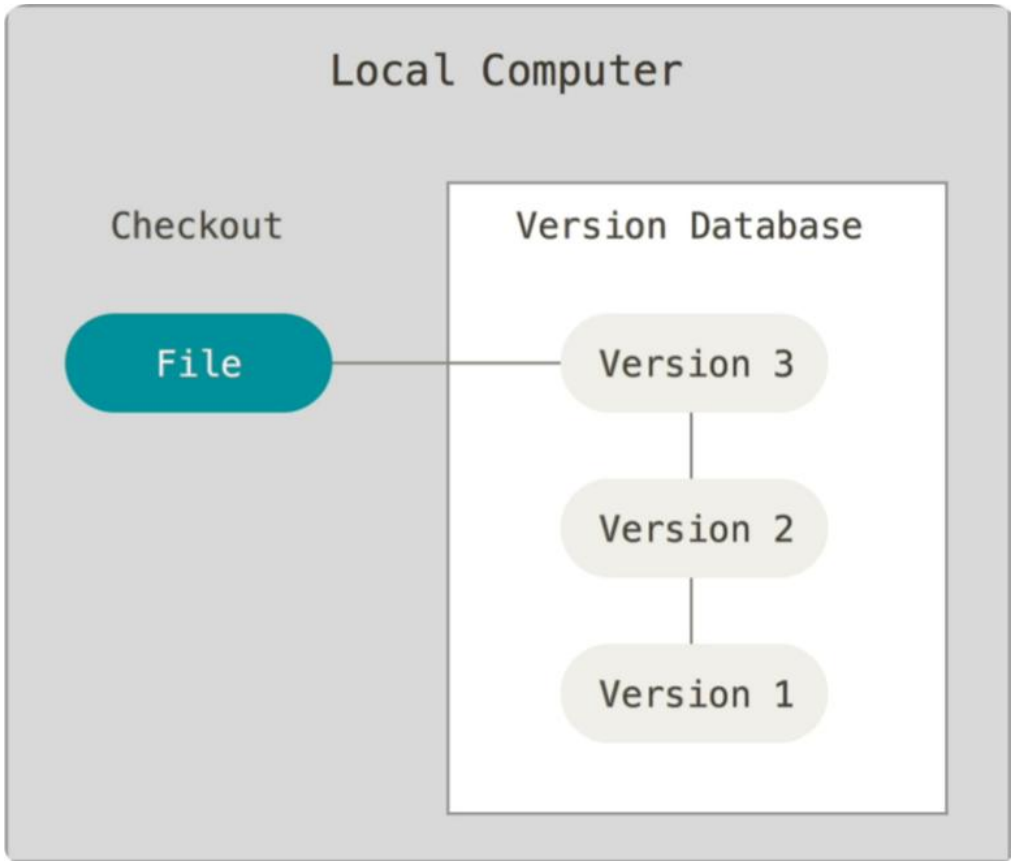
Benefits of Version Control

- compare file versions
- identify and merge changes
- track builds across environments (development, QA, production)
- support onboarding of new developers
- maintain independent code versions
- simplify debugging by comparing versions
- allow rollback to previous working versions

Main Features

Repository

- collection of files under version control



Clone (Checkout)

- copy of the repository to a local machine

Commit

- recorded change to files
- creates a changeset with a unique revision

Changeset

- includes:
    - changes made
    - author
    - timestamp
    - message explaining the change

Commit Messages

- explain why a change was made

- stored in the VCS, not the file

Revision History

- complete history of all changes
- stored across multiple systems
- supports recovery and auditing

Markers (Tags)

- reference a specific version of the system
- used for releases or major updates

Accountability

- every change is linked to a user

Coding Guidelines and Standards

Purpose

Ensure software is:

- safe
- secure
- reliable
- testable
- maintainable
- portable

Why they matter

- prevent defects
- ensure consistency
- reduce risk of security issues
- improve team collaboration

Key Drivers

- compliance with standards (for example ISO)
- consistent code quality
- built-in security

- reduced development cost and faster delivery

Programming Tools

Compilers

- convert source code into machine code
- example: C# or Java compilation

Interpreters

- execute code directly without full compilation
- may:
    - interpret directly
    - use intermediate representation
    - execute precompiled code

Debuggers

- detect and fix errors
- features include:
    - step-by-step execution
    - breakpoints
    - variable tracking
    - runtime modification

GUI Designers

- tools for building interfaces visually
- use drag-and-drop (WYSIWYG)

Peer Reviews

Purpose

- detect and correct defects early
- prevent issues reaching production

Benefits

- early bug detection
- improved code quality

- knowledge sharing
- consistent design
- improved security
- stronger team collaboration
- increased stakeholder confidence

Retrospectives

What they are

Meetings held at the end of an iteration.

Purpose

- reflect on what happened
- identify improvements

Outcome

- continuous improvement in process and performance

Key Takeaways

- Version control tracks and manages changes across a project
- Enables safe concurrent development
- Repositories, commits, and changesets form the core workflow
- Traceability links work across the lifecycle for visibility and compliance
- Version control improves collaboration, debugging, and recovery
- Coding standards ensure consistency, quality, and security
- Programming tools support development, execution, and debugging
- Peer reviews and retrospectives improve quality and team performance

Testing

Why Testing is Necessary

Testing ensures that software:

- meets requirements
- functions correctly
- performs reliably
- is safe for use

Key purposes:

- detect defects before release
- reduce risk
- validate system behaviour
- ensure quality

Early Testing

Why start early

- Defects can occur at any stage of the SDLC
- Earlier detection reduces cost and impact

Examples of early testing

- design reviews
- unit testing
- regression testing

Key Idea

Testing should begin as early as possible to minimise later-stage defects.

Risk-Based Testing

What is risk

A factor that may cause negative consequences, defined by:

- likelihood
- impact

Risk-based approach

- identify risks early
- use risks to guide testing

Used to:

- choose test techniques
- determine level of testing
- prioritise critical areas
- reduce risk through testing and non-testing activities

Conformance to Requirements

Conformance

Degree to which software meets predefined requirements.

Includes

- functional requirements (what the system does)
- non-functional requirements (how well it performs)

Key Idea

Testing verifies whether the system adheres to both types.

Finding Defects

- Defects originate from human error
- Testers aim to expose defects before production

Roles

- developers may see defects as delays
- testers assume defects exist and aim to find them

Goal

Trigger failures to reveal hidden defects.

Testing vs Debugging

Testing

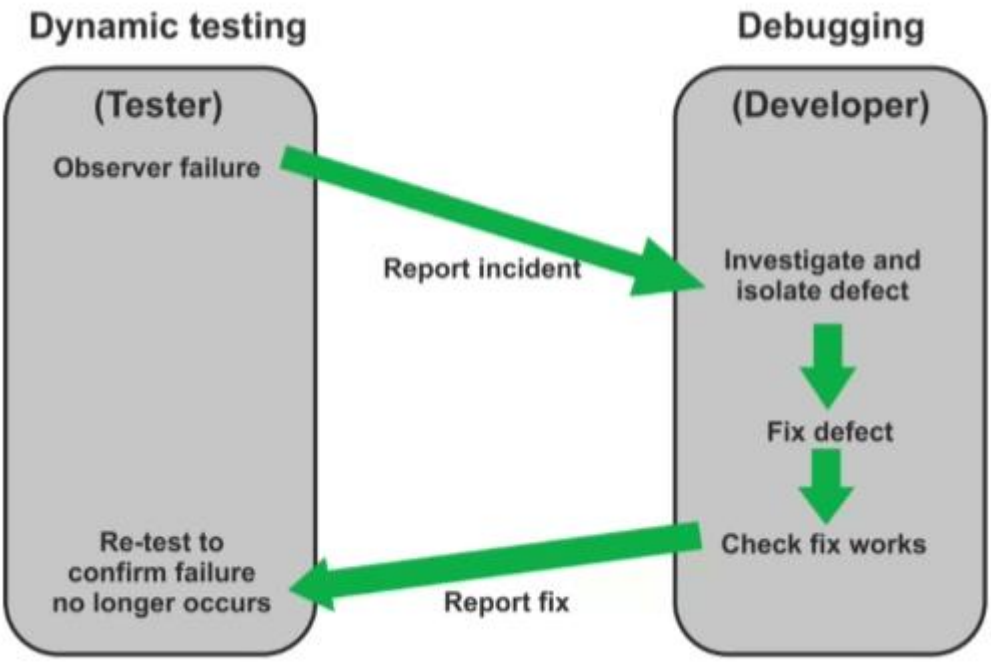
- identifies failures
- reveals presence of defects

Debugging

- identifies root cause
- fixes the defect
- verifies the fix

Process

Testing → failure → debugging → fix → re-testing



Levels of Testing

Unit Testing

- tests individual components
- smallest testable unit
- validates expected behaviour

Integration Testing

- tests combined components
- identifies interaction issues

Tools:

- test drivers

- test stubs

System Testing

- tests complete system
- checks compliance with requirements

Acceptance Testing

- validates system against business needs
- ensures system is fit for purpose
- focuses on readiness for deployment

Test Techniques

Functional Testing

- tests what the system does
- based on requirements
- black-box approach

Non-Functional Testing

- tests how well the system performs

Includes:

- security
- performance
- usability
- reliability

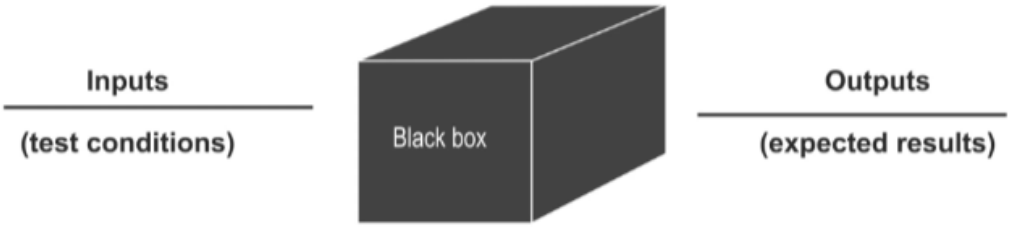
Regression Testing

- re-tests system after changes
- ensures no new defects introduced

Black Box Testing (Specification-Based)

What it is

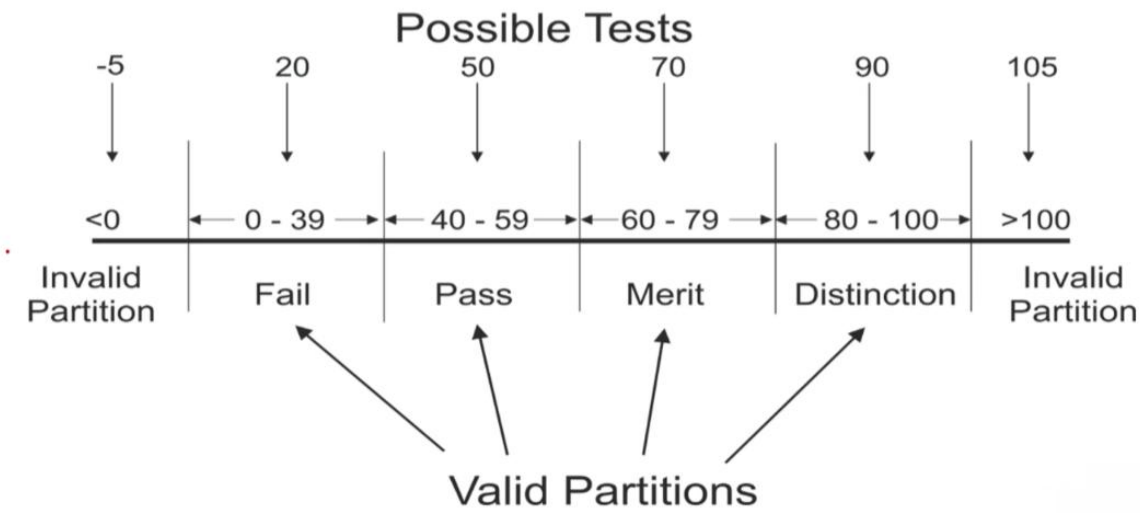
- tests external behaviour
- no knowledge of internal structure



Techniques

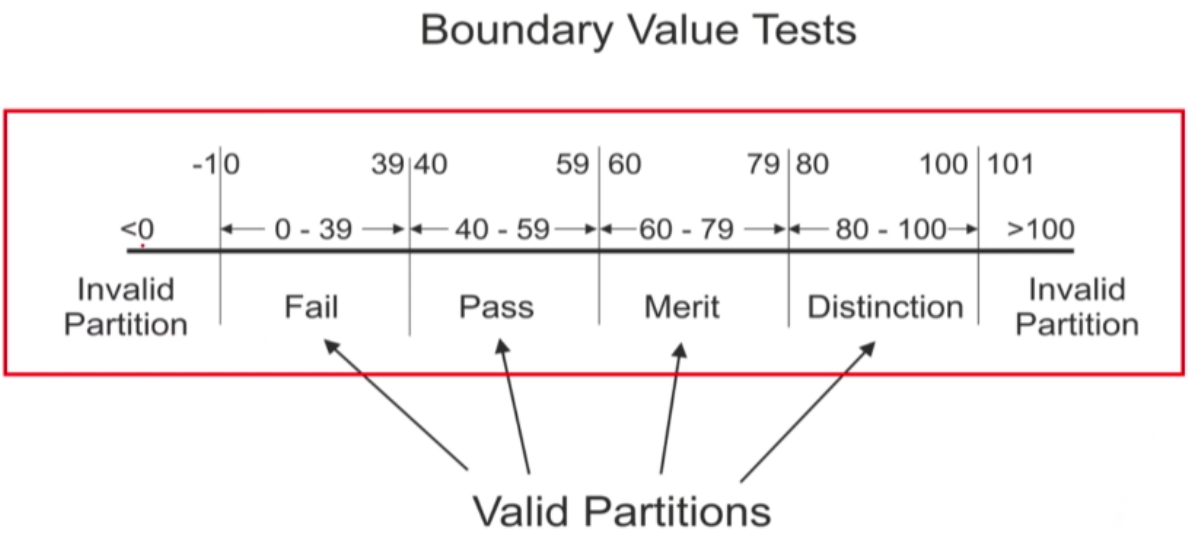
Equivalence Partitioning

- divide inputs into groups with similar behaviour
- test one value per group



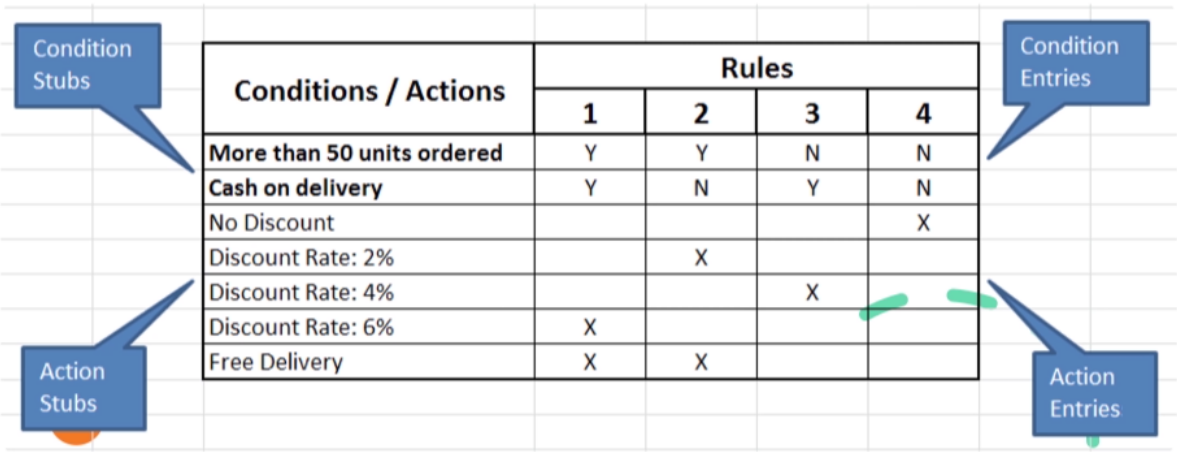
Boundary Value Analysis

- test edge values of partitions
- higher chance of defects at boundaries



Decision Tables

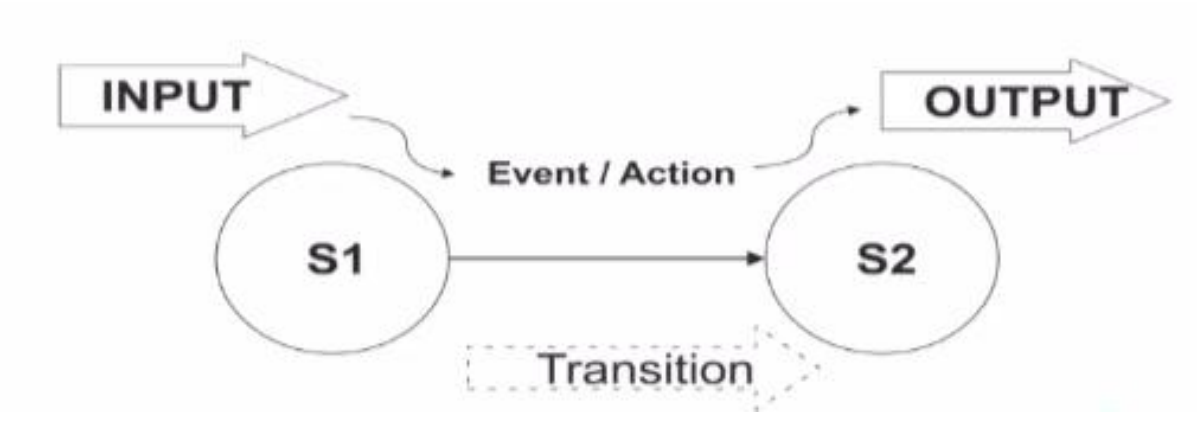
- represent combinations of conditions and actions
- used for complex business rules



Conditions / Actions	Rules			
	1	2	3	4
More than 50 units ordered	Y	Y	N	N
Cash on delivery	Y	N	Y	N
No Discount				X
Discount Rate: 2%		X		
Discount Rate: 4%			X	
Discount Rate: 6%	X			
Free Delivery	X	X		

State / Transition Diagrams

- represent system states
- show transitions between states
- identify events and outcomes



Experience-Based Testing

What it is

- based on tester experience and intuition

When used

- limited time or resources
- complex or poorly documented systems

Focus areas

- critical functions
- commonly used features
- areas likely to fail

Functional vs Non-Functional Tests

Functional Tests

- unit testing
- smoke testing
- sanity testing
- integration testing
- interface testing
- system testing
- regression testing
- user acceptance testing

Non-Functional Tests

- availability testing
- compatibility testing
- compliance testing
- configuration testing
- endurance testing
- ergonomics testing
- interoperability testing
- installation testing
- load testing
- localisation and internationalisation testing
- maintainability testing
- operational readiness testing
- performance testing
- recovery testing
- reliability testing
- resilience testing
- security testing
- scalability testing
- stress testing
- usability testing
- volume testing

Approaches to Testing

Different organisations adopt different approaches based on:

- methodology
- project type

- organisational preferences

Examples

- risk-based approach
- model-based approach
- heuristic approach
- consultative approach
- methodical approach
- standards-based approach

Independent Testing Teams

What it is

Testing performed by a separate team independent from developers.

Benefits

- unbiased perspective
- better defect detection
- no assumptions about quality
- focuses on actual system behaviour

Disadvantages

- communication gaps
- potential outdated documentation
- delays due to late involvement
- developers may rely too heavily on testers

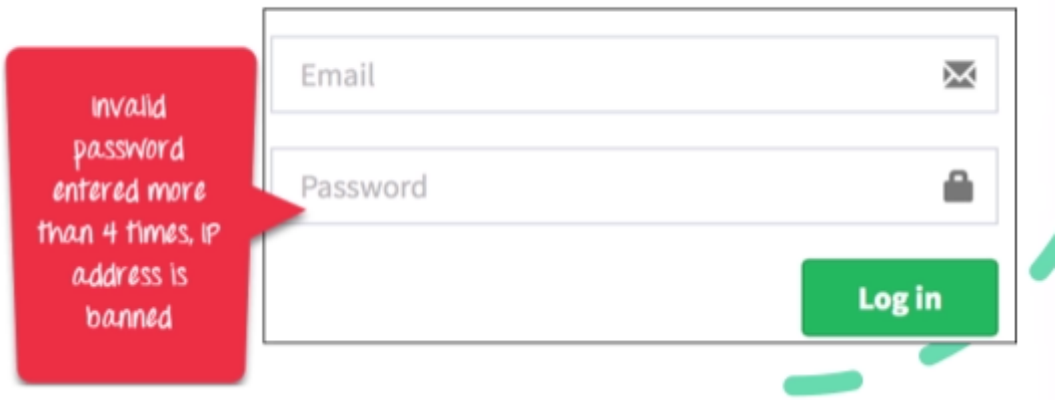
Equivalence Classes and Use Cases

Equivalence Partitioning

- divides input data into groups
- each group tested at least once

Use Case Testing

- tests full system behaviour
- follows real user interactions from start to finish



Debugging

Efficient Debugging Practices

- identify type of error
- reproduce issue
- isolate cause
- apply fix
- verify solution

Types of Errors

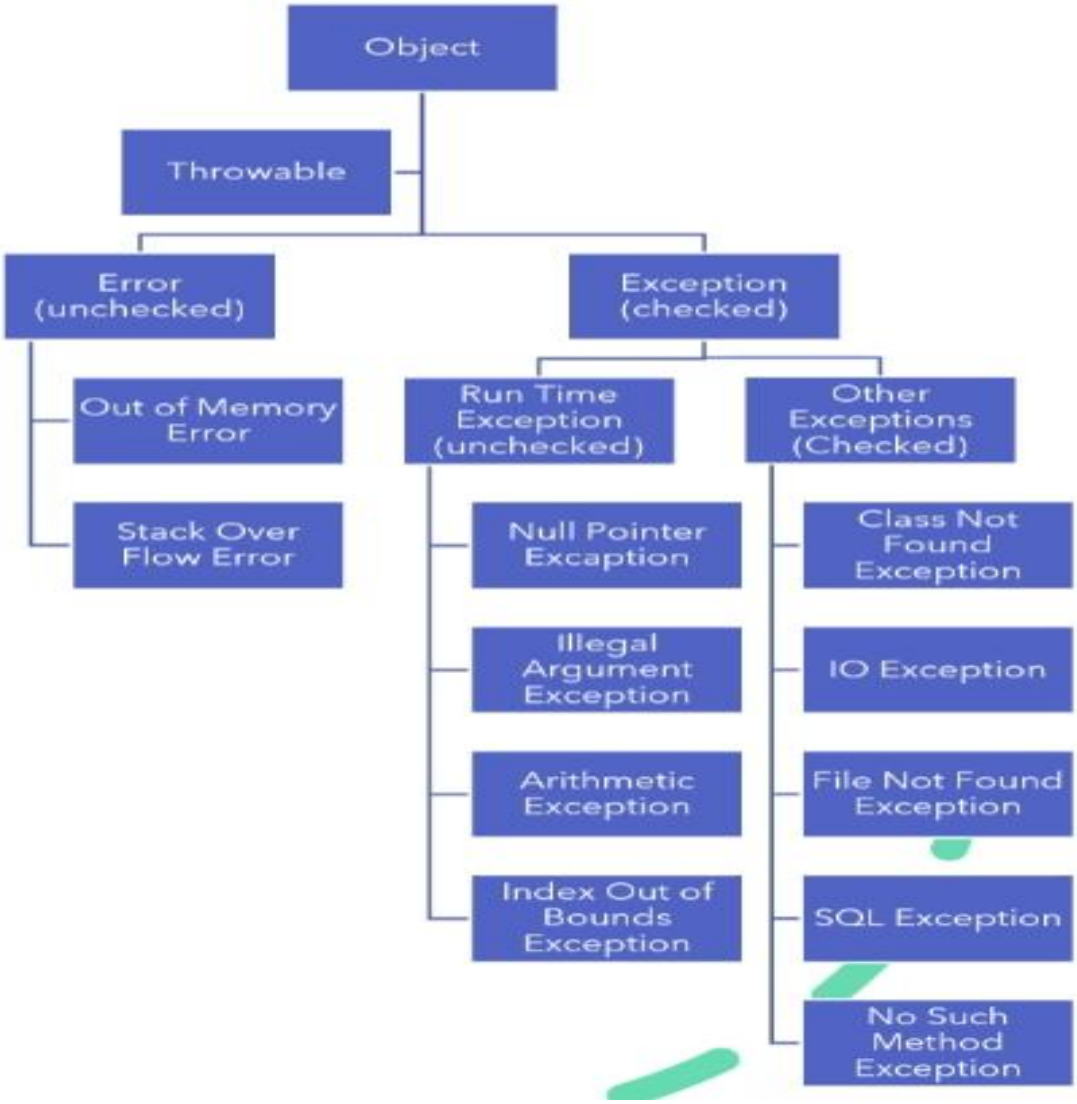
- Syntax errors
- Semantic errors
- Logical errors

Detection Timing

- Compile-time errors
- Runtime errors

Exception Handling

- handles unexpected conditions
- maintains program flow
- uses language constructs or system mechanisms



Reproduction

- recreate issue using defined steps
- essential for diagnosing bugs
- improves reliability of fixes

Elimination

- identify and remove defects early

- reduce propagation of issues

Logging

- record defects and system behaviour
- track issues and fixes
- support debugging and maintenance

A step by step approach towards Defect Logging and Tracking

Popular Industry Logging Standard

https://www.servicenow.com/uk/

https://learning.servicenow.com/now/lxp/home

Test Coverage

What it is

A metric measuring how much of the system is tested.

Expressed as

- percentage of code executed

Types of Coverage

- statement coverage
- decision coverage
- condition coverage
- multiple condition coverage
- path coverage

Purpose

- identify untested areas
- improve test completeness
- support quality assurance

Key Takeaways

- Testing ensures software meets requirements and reduces risk
- Early testing minimises cost and impact of defects
- Risk-based testing prioritises critical areas
- Testing identifies failures, debugging fixes them
- Four levels of testing: unit, integration, system, acceptance
- Functional tests check behaviour, non-functional tests check performance
- Black-box techniques include equivalence partitioning and boundary analysis
- Experience-based testing relies on expertise and intuition
- Test coverage measures completeness of testing
- Independent testing improves objectivity but may impact communication

Deployment

What it is

Deployment is the process of preparing, releasing, and making software operational in a target environment.

It includes:

- installation
- configuration
- testing in production
- optimisation

Activities During Release

- implementation planning
- training users and operations
- preparing user documentation
- building the software release
- deploying the software

Implementation Planning

A complete implementation plan includes:

- work plan
- resources and budget
- stakeholders
- risk assessment
- quality control

Purpose

- ensure smooth transition to production
- minimise disruption
- manage risk effectively

Training Users and Operations

- ensures users can operate the system correctly
- prepares operational teams to support the system
- reduces errors and resistance to adoption

User Documentation

What it is

Documentation provided to end users to help them use the system.

Purpose

- support correct usage
- reduce training requirements
- improve user experience

Types

- user manuals
- training materials

Build and Deploy Software

https://www.selenium.dev/

https://cucumber.io/

Build

- compile and package the software
- prepare release artefacts

Deploy

- install software in target environment
- configure system settings
- test and optimise performance

Configuration Management (CM)

What it is

A process for maintaining consistency between:

- system requirements
- design
- performance
- operational behaviour

Purpose

- control system changes
- maintain system integrity
- ensure traceability

Benefits

- improves performance and reliability
- extends system lifespan
- reduces cost and risk
- supports defect correction
- enables controlled system evolution

Deployment Strategies

Parallel Run

- new and old systems run together
- reduces risk
- allows comparison

Big Bang Deployment

- full system switch at once
- old system removed immediately

Advantages

- fast implementation
- no need to maintain two systems

Disadvantages

- high risk
- difficult to recover if issues occur

Phased Deployment

- system introduced gradually
- users or modules migrated over time

Advantages

- lower risk
- easier issue isolation

Disadvantages

- slower implementation
- more complex coordination

Early Life Support (ELS)

What it is

Post-deployment support period for a new system.

Based on

ITIL (Information Technology Infrastructure Library)

Purpose

- monitor system performance
- manage incidents and issues
- adjust service levels and thresholds

Developer Role

- identify and resolve issues
- support system stabilisation

Alpha and Beta Releases

Alpha

- early testing phase
- tested internally
- uses white-box and internal black-box testing

Beta

- feature-complete but may contain defects
- released to limited users
- focuses on usability and real-world testing

Major and Minor Releases

Versioning System

Format: X.Y.Z

- Major
- Minor
- Patch

Major Version

- significant or breaking changes

Minor Version

- new features added

Patch Version

- bug fixes and small updates

Software Upgrades

What it is

A major update that introduces significant changes.

Characteristics

- replaces previous version
- may include new features or architecture changes
- may not be backward compatible

Software Licensing

What it is

Defines how software can be used, distributed, and modified.

Types of Licences

Proprietary Licence

- owned by a company
- restricted use and modification

End-User Licence (EULA)

- legal agreement between user and provider
- defines usage terms

Enterprise Licence

- allows organisation-wide usage
- may include restrictions

Site Licence

- allows installation across multiple systems at one location

---

Workstation Licence

- limited to a single machine

Perpetual Licence

- one-time purchase
- indefinite use of a specific version

Open-Source Licence

- allows modification and redistribution
- may be free or paid
- subject to defined conditions

Key Takeaways

- Deployment makes software operational in a live environment
- Implementation planning reduces risk and ensures smooth release
- User training and documentation improve adoption and usability
- Configuration management ensures consistency and control
- Deployment strategies include parallel, phased, and big bang approaches
- Early life support stabilises systems after release
- Versioning tracks software changes using major, minor, and patch numbers
- Licensing defines how software can be used and distributed

Maintenance

What it is

Maintenance is the process of modifying and supporting software after deployment to ensure it remains:

- functional
- secure
- efficient
- relevant

Types of Maintenance

Adaptive

- Modify system to cope with environment changes
- Examples:
    - OS updates
    - DBMS changes

Perfective

- Add or improve functionality
- Based on new or changed user requirements

Corrective

- Fix defects and errors
- Often identified by users or testing

Preventive

- Improve maintainability and reliability
- Reduce likelihood of future issues

Maintenance Activities

Major Enhancements

- Introduce significant new functionality
- May change system scope or direction

Minor Enhancements

- Add moderate improvements
- Do not change overall system direction
- Example: version change from 3.1 to 3.2

Error Correction

- Fix defects in modules or components
- Address security issues
- Correct requirement errors or omissions

Temporary Workarounds

- Short-term fixes to bypass issues
- Not a permanent solution
- Often implemented as patches

Performance Improvement

Examples:

- buffer space optimisation
- disk allocation
- memory usage
- I/O priority

Purpose

- ensure performance requirements are met
- identify bottlenecks
- improve system efficiency

Routine Updates

Software Updates

- small, incremental improvements
- include:
    - bug fixes
    - security patches
    - compatibility updates

Importance of Updates

- protect against security risks
- improve performance
- extend system lifespan
- fix defects
- introduce minor features

Reasons for Change

- changing user requirements
- incidents or problems
- regulatory changes
- infrastructure or platform changes

Changed User Requirements

- requirements evolve over time
- business processes change

Problems and Incidents

- incident: interruption of service
- problem: recurring issue causing incidents

Regulatory Changes

- systems must comply with laws and standards
- requires documentation and validation

Includes:

- system specifications
- verification and validation records
- process assurance evidence

Infrastructure Changes

Examples:

- OS changes (Windows to Linux)
- hardware to cloud migration
- API additions
- database changes

Service Level Agreement (SLA)

What it is

Agreement between service provider and customer defining expected service levels.

Includes

- performance metrics
- service expectations
- penalties for non-compliance

Maintenance Processes

- reporting
- logging
- incident management
- problem management

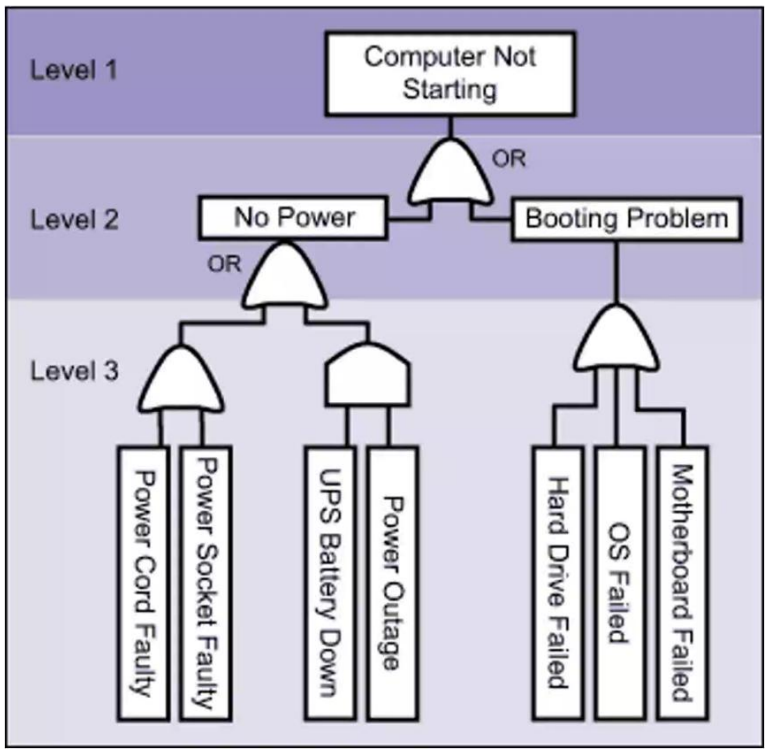
- first-line support
- second-line support

Problem Solving Techniques

- brainstorming
- fault tree analysis
- Kepner-Tregoe analysis
- Ishikawa diagrams
- decomposition

Fault Tree Analysis

- top-down approach
- identifies root causes of defects
- breaks causes into smaller events

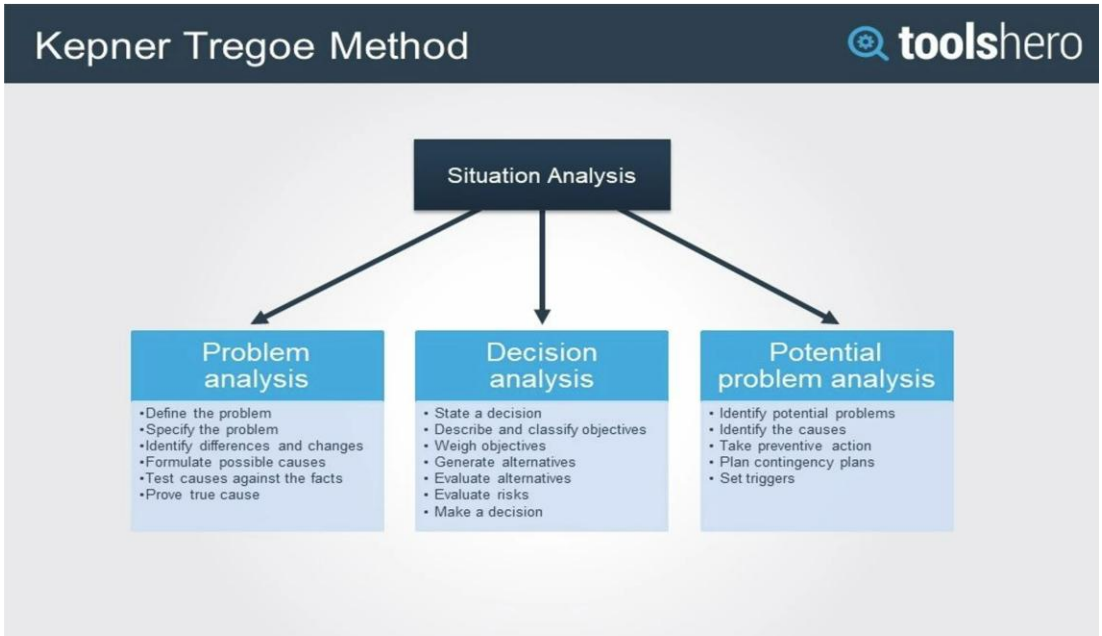


Kepner-Tregoe (KT)

- structured problem-solving method

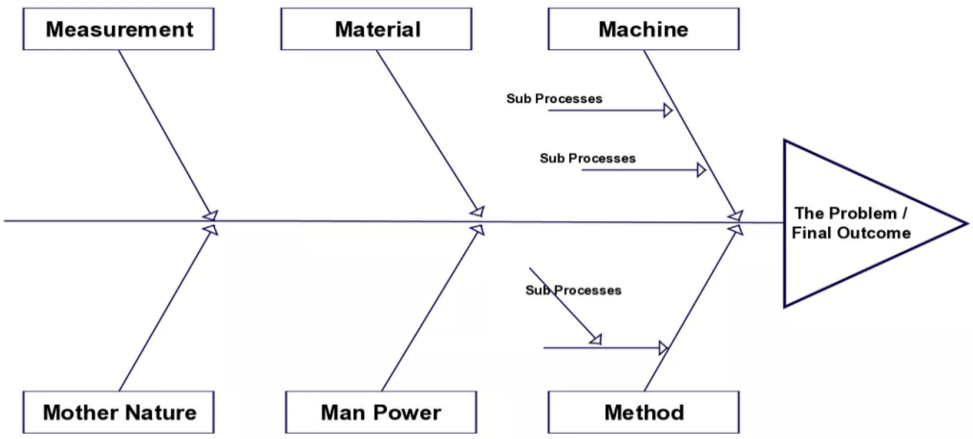
Key questions:

- what happened
- why it happened
- how to act
- future outcome



Ishikawa Diagram

- visual cause-and-effect diagram
- identifies root causes



Decomposition

- breaks system into smaller components

Benefits:

- simplifies complexity
- improves focus
- supports parallel development

Importance of Controlling Change

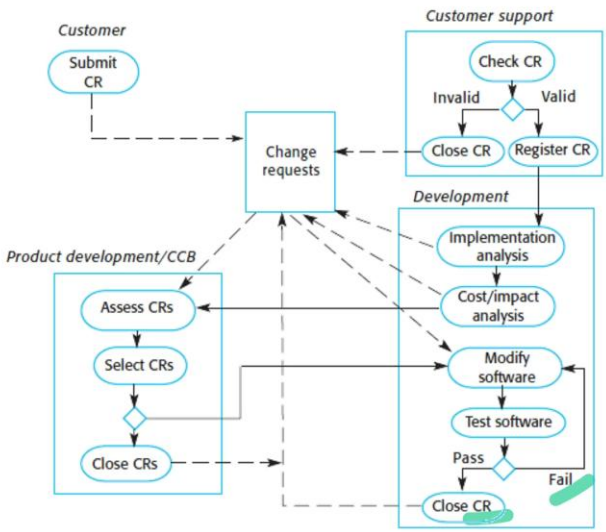
- change is inevitable in software systems
- must be controlled to reduce risk

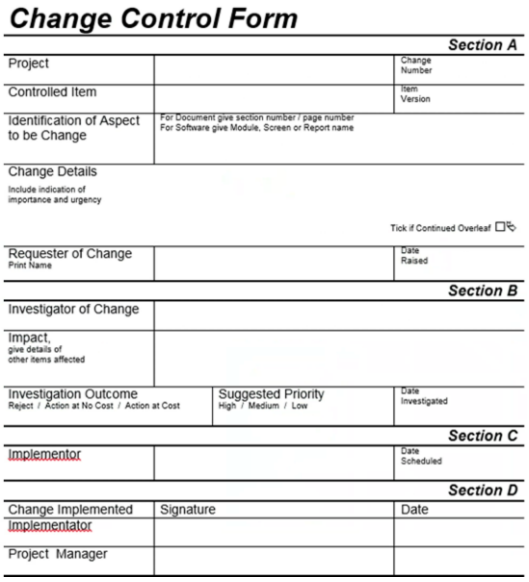
Change Management

- evaluates cost vs benefit
- approves necessary changes
- tracks system modifications

Change Process Flow

1. Request for Change (RFC)
2. Change assessment and planning
3. Approval
4. Implementation





Change Control Form

Section A		
Project		Change Number
Controlled Item		Item Version
Identification of Aspect to be Change	For Document give section number / page number For Software give Module, Screen or Report name	
Change Details Include indication of importance and urgency		
		Tick if Continued Overleaf ☐
Requester of Change Print Name		Date Raised
Section B		
Investigator of Change		
Impact, give details of other items affected		
Investigation Outcome Reject / Action at No Cost / Action at Cost	Suggested Priority High / Medium / Low	Date Investigated
Section C		
Implementor		Date Scheduled
Section D		
Change Implemented Implementator	Signature	Date
Project Manager		

Software Development Plan

What it is

A comprehensive document for managing the project.

Core Components

- scope statement
- critical success factors
- deliverables
- work breakdown structure
- schedule
- budget
- quality plan
- human resources plan
- stakeholder list
- communication plan
- risk register
- procurement plan

Return on Investment (ROI)

What it is

A financial measure of project value.

Formula

ROI = (Present Value of Benefits / Present Value of Costs) × 100

Purpose

- evaluate profitability
- justify development investment

Traceability Matrix

What it is

A document linking related project artefacts.

Purpose

- ensure requirements are met
- track coverage

Sno	Req ID	Req Desc	TC ID	TC Desc	Test Design	Test Designer	UAT Test Req?	Test Execution			Defects?	Defect ID	Defect Status	Req Coverage Status
								Test Env	UAT Env	Prod Env				
1	Req01	Login to the Application	TC01	Login with Invalid Username and valid password	Com									
2			TC02	Login with Valid Username and invalid password	Com									
3			TC03	Login with valid credentials	Com									

	Requirement #1	Requirement #2	Requirement #3	Requirement #4	Requirement #5	Requirement #6	Requirement #7	Requirement #8	Requirement #9	Requirement #10	Requirement #11	Requirement #12	Requirement #13
Test Case #1			X				X				X		
Test Case #2		X		X			X						X
Test Case #3	X			X			X			X			
Test Case #4				X	X								
Test Case #5								X					
Test Case #6					X				X				

Types of Traceability

Forward

- ensures requirements are implemented and tested

Backward

- ensures no unnecessary features are added
- prevents scope creep

Bi-Directional

- combines forward and backward
- tracks coverage and impact of changes

Deployment Plan

What it is

Defines how deployment will be carried out.

Includes

- system support details
- issue tracking
- escalation processes
- roles and responsibilities

Key Takeaways

- Maintenance ensures software remains functional and relevant after deployment
- Four types: adaptive, perfective, corrective, preventive
- Updates and enhancements improve performance, security, and usability
- Changes are driven by users, incidents, regulations, and infrastructure
- Change management controls system evolution
- Traceability ensures requirements are implemented and tested
- ROI helps justify software investment
- Maintenance relies on structured processes and problem-solving techniques

Methodologies

What are Methodologies

Software development methodologies are structured approaches used to plan, manage, and deliver software projects.

They define:

- workflow
- collaboration
- planning
- delivery approach
- handling of change

Types of Methodologies

Iterative Methodologies

Development occurs in repeated cycles with continuous improvement.

Examples:

- Agile
- Scrum
- XP
- RAD
- Crystal
- DSDM
- Lean
- JAD

Sequential Methodologies

Development follows fixed stages in order.

Examples:

- Waterfall

Hybrid Approaches

Combine iterative and sequential characteristics.

Examples:

- Spiral
- FDD
- SDLC
- RUP

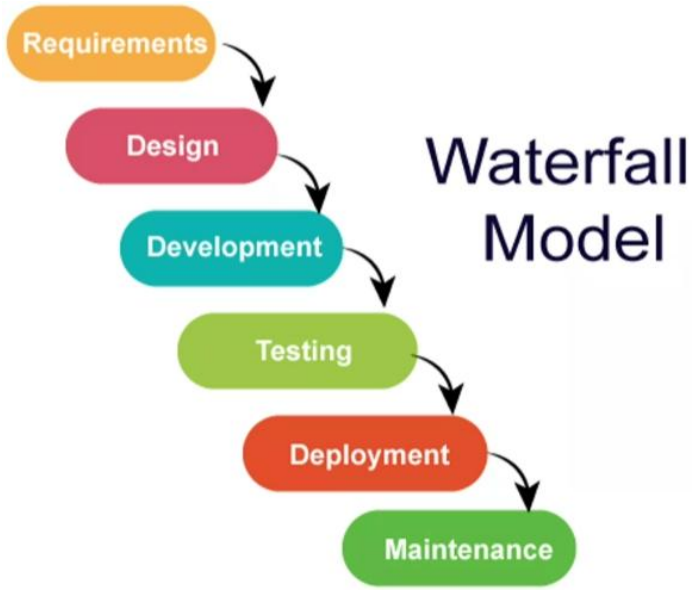
Waterfall Methodology

What it is

A sequential development process where each phase is completed before the next begins.

Characteristics

- structured stages
- strong documentation
- limited flexibility after planning



Process Flow

Requirements → Design → Development → Testing → Deployment → Maintenance

---

Advantages

- clearer early project definition
- easier to estimate scope and deliverables
- suitable for projects with stable requirements
- useful where extensive design is required
- supports management of large dependencies

Disadvantages

- difficult to adapt to changing requirements
- defects often discovered late
- communication risks are higher
- changes later in development are expensive
- risk of assumptions being incorrect early on

Agile Methodology

What it is

An iterative and incremental approach focused on flexibility, collaboration, and continuous delivery.

Core Idea

Software evolves through repeated cycles called iterations or sprints.

Agile Process

- start with simple design
- develop small modules
- test frequently
- gather feedback
- improve continuously

Agile Values

- individuals and interactions over processes and tools
- working software over documentation
- customer collaboration over contract negotiation
- responding to change over following a plan

Advantages

- faster feedback
- greater flexibility
- earlier defect discovery
- stronger collaboration
- reduced rework
- continuous improvement
- improved visibility through boards and charts

Disadvantages

- difficult to estimate long-term outcomes
- requires strong communication
- intensive for developers and stakeholders
- less predictable final deliverable
- difficult on large or distributed projects
- weaker handling of dependencies and deadlines

Scrum

What it is

An Agile process framework.

Key Features

- small cross-functional teams

- self-organising teams
- prioritised backlog
- fixed-length sprints (1–4 weeks)
- potentially shippable product after each sprint
- retrospectives after iterations

Scrum Activities

- sprint planning
- daily stand-ups
- sprint review
- retrospective

Kanban

What it is

A visual workflow management method.

Key Principles

- visualise workflow using boards/cards
- limit work in progress (WIP)
- optimise lead time

Benefits

- improved visibility
- smoother workflow
- reduced bottlenecks

Extreme Programming (XP)

What it is

An Agile methodology focused on:

- software quality
- rapid feedback
- responsiveness to change

Characteristics

- short development cycles
- frequent releases
- strong testing practices

Test-Driven Development (TDD)

What it is

A development approach where tests are written before code.

TDD Cycle

1. Add a test
2. Run tests and fail
3. Write code
4. Run tests again
5. Refactor code
6. Repeat

Benefits

- improved code quality
- better test coverage
- safer refactoring
- clearer requirements

Unified Process (UP)

What it is

An iterative and incremental framework focused on architecture and use cases.

Characteristics

- iterative
- architecture-centric
- use-case driven

Refinement

Rational Unified Process (RUP) is the best-known implementation.

Unified Process Phases

Inception

- gather requirements
- assess feasibility
- analyse risks and cost

Elaboration

- refine requirements
- create architecture
- prepare business case

Construction

- develop and implement features
- produce working iterations

Transition

- deploy releases
- fix defects
- deliver builds to users

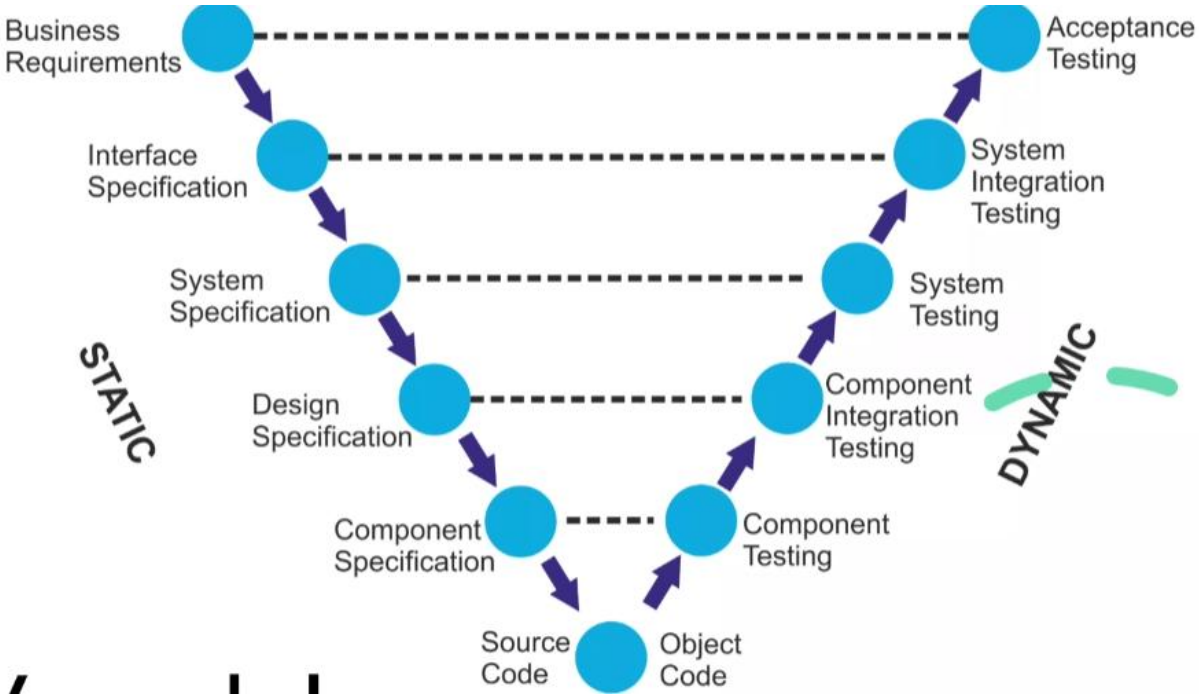
V-Model

What it is

A sequential model linking development stages with corresponding testing stages.

Key Idea

Every development activity has a matching testing activity.



Spiral Model

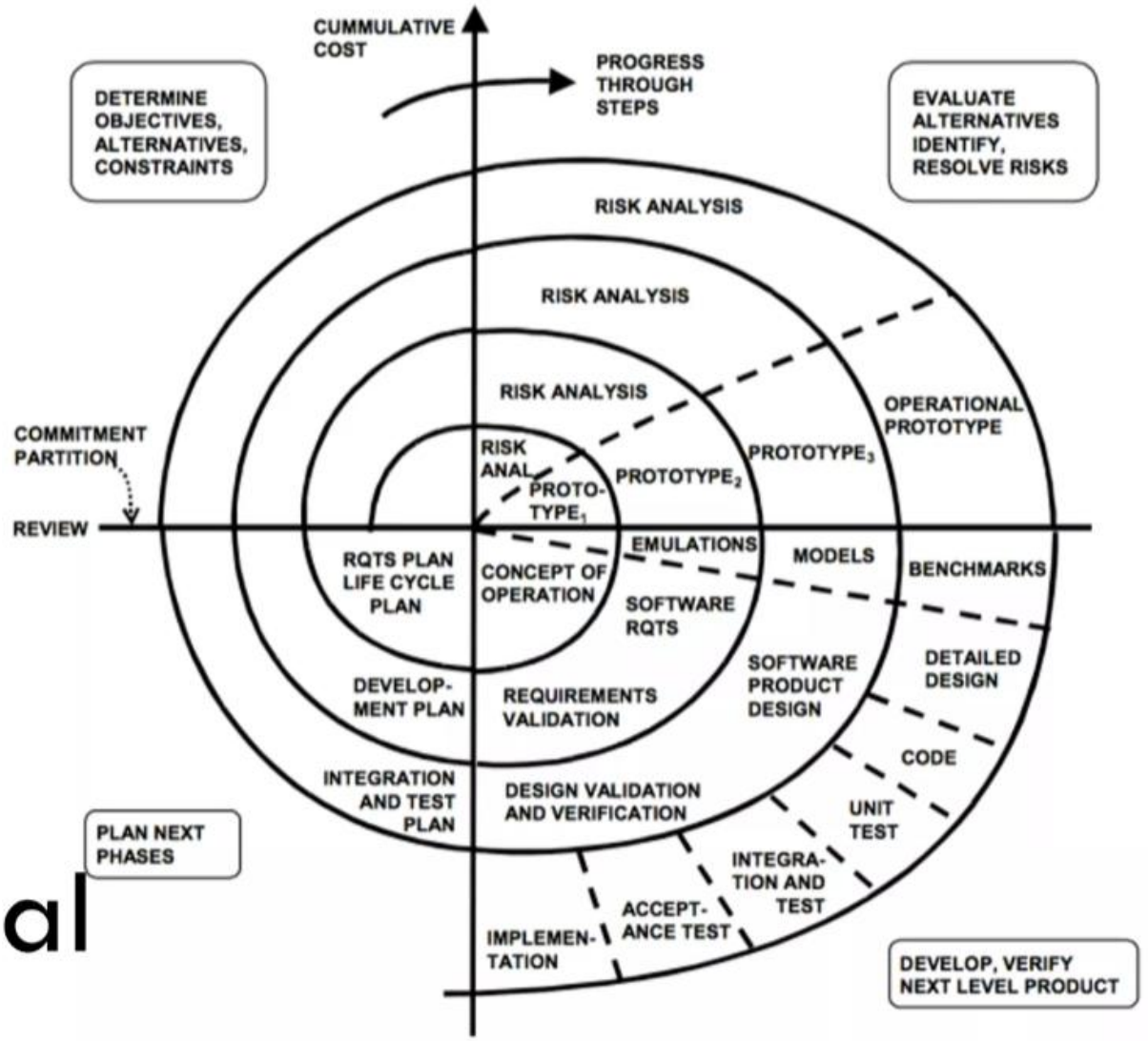
What it is

A risk-driven methodology combining:

- iterative development
- sequential structure

Focus

- risk analysis
- repeated refinement



Lean Development

What it is

Adaptation of lean manufacturing principles to software development.

Focus Areas

- reducing waste
- improving efficiency
- delivering value quickly
- continuous improvement

DevOps

What it is

A set of practices combining:

- software development (Dev)
- IT operations (Ops)

Goals

- shorten development lifecycle
- automate processes
- deliver updates frequently
- align development with business objectives

Other Methodologies

Crystal Methods

- lightweight Agile family
- focuses on people and communication

DSDM

- Agile framework
- emphasises rapid delivery and user involvement

FDD (Feature Driven Development)

- organised around features
- combines iterative and sequential practices

RAD (Rapid Application Development)

- prioritises rapid prototyping and quick delivery

JAD (Joint Application Development)

- focuses on collaborative requirements gathering

Choosing a Methodology

Selection depends on:

- project size
- complexity
- risk level
- stakeholder involvement
- requirement stability
- delivery deadlines

Key Takeaways

- Methodologies define how software projects are managed and delivered
- Iterative approaches support flexibility and continuous feedback
- Sequential approaches prioritise structure and planning
- Agile focuses on collaboration and adaptability
- Waterfall follows fixed stages with strong upfront planning
- Scrum and Kanban are common Agile process tools
- XP and TDD emphasise software quality and testing
- DevOps combines development and operations for faster delivery
- Different methodologies suit different project types and organisational needs